Bridging Repairability Gaps in Shared Bus Architecture with Shared Physical Memory Implementation

Nikhil Karkare
Google India Pvt. Ltd.
Bengaluru, India
nikhilsk@google.com

Wilson Pradeep
Google India Pvt. Ltd.
Bengaluru, India
wpradeep@google.com

***Abstract* – Power, performance and area (PPA) are the three key differentiating factors in today's SoCs (System on-chips). To achieve best-in class PPA, complex processor cores using shared bus architecture typically use single physical memory across multiple logical memories to improve overall design efficiency. However, such an implementation imposes several limitations on enabling redundancy and repair support using on-chip memory test, redundancy analysis and repair implementation for the full range of memory space, resulting in a degraded repair coverage. This in turn could result in inferior yield and thereby increase cost margin for the product. In this paper, we propose a few novel strategies to seamlessly enable redundancy sharing across logical memories shared by a single physical memory to mitigate the limitation with conventional approaches. Additional schemes are proposed to resolve potential conflict conditions that could arise as a result of this redundancy sharing. Furthermore a comprehensive fault injection and repair verification methodology is shared to assess the efficacy of proposed methods. The experimental results by using the proposed solutions on two sample clusters on a large-scale design using shared bus architecture bridged a gap of 5-6% drop in repair coverage due to the shared physical memory implementation.**

***Keywords* – MBIST, Shared Bus MBIST, Memory Repair, BIRA, BISR, Yield, Fault Injection**

I. INTRODUCTION

The steep rise in the foot print dominance of embedded memories in today's SoCs [1] has laid a great emphasis on effective testing and repairability of these components to improve the overall product yield [2][3]. Implementing embedded memories with in-built redundancies [4] is the most common method used to achieve this. The provision of replacing faulty elements with redundant elements provides additional fault tolerance. Row and/or column redundancy are common types of redundant elements typically in use.

As a current trend, most SoCs deploy on-chip solutions for redundancy analysis and repair implementation [5] as they are highly cost effective compared to an external ATE (Automatic Test Equipment) solution.

Recent high performance processor cores typically implement shared bus architecture for memories. It mitigates the limitations imposed by conventional BIST (Built-in Self Test) [6] by providing a common access path to memories in both functional and test modes.

Memory compilers usually provide redundancy details (spare rows and columns) of each physical memory instance in a library file. In shared bus architecture, the abstraction level of memory BIST insertion, test and repair is at logical memory (LM) level which can include one or more physical memories (PM). LM represents a complete address space as seen from outside the shared bus interface. Hence, to enable repair implementation, the redundancy information of each PM needs to be exported to its corresponding LM level in the form of a library map file. BIST insertion tool uses this mapping information to insert the appropriate self-repair chain and register to drive the repair ports of memories.

In recent ultra deep sub-micron (<10nm) technologies, analysis shows that usage of a single large composite PM instance for a cluster of LMs gives significant power and area benefits than using a group of smaller PM entities.

While the shared PM scenarios give considerable PPA benefit, it does impose some limitations and challenges with regard to repair implementation which could result in notable yield impact.

In this paper, we present a few novel strategies to address the repairability gap in case of different shared PM scenarios to mitigate potential yield impact. The proposed techniques primarily focus on the following.

a) Implementing a simple timing-friendly custom logic outside the memory instance/wrappers to enable parallel redundancy on each LM associated with the single PM.
b) Avoid potential repair conflict scenarios between LMs using logic based voting mechanisms.

We also share a comprehensive fault injection and repair verification flow that can be seamlessly deployed to evaluate the efficiency of the repair solution using proposed custom methods. The rest of this paper is organized as follows. Section II discusses a) overview of shared bus architecture and repair solutions and b) scenarios of shared PM which impact repair. Section III details the proposed technique used to address the repairability gap in shared PM use cases. Section IV summarizes the proposed comprehensive fault injection and repair verification methodology adopted. Section V shares few experimental results, while Section VI concludes.

2023 IEEE International Test Conference India (ITC India) | 979-8-3503-1214-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ITCINDIA59034.2023.10235393

Authorized licensed use limited to: Uppsala Universitetsbibliotek. Downloaded on April 06,2026 at 14:30:24 UTC from IEEE Xplore. Restrictions apply.

II. BACKGROUND

In recent designs, embedded memories usually tend to be more fault prone compared to standard logic primarily due to their high density implementation and dominant share in the design area. Hence, efficient test and repair of memories is extremely critical for overall yield improvement. Redundancy is a common technique used by most memory compilers to repair hard errors in memories. Conventional approach involves including spare columns(s) and/or row array of memory cells along with the regular memory cells. Modern memory test solutions usually deploy BISR (Built-in Self Repair) schemes which use on-chip BIRA (Built-in Redundancy Analysis) engines to perform the redundancy analysis [7][8][9] based on memory test data in the event of failures. The generated repair information such as row/column repair enable (*RRE/CRE*) and fault row/column address (*FCA/FRA*) are generated and stored in on-chip fuses which get loaded onto the BISR registers driving memories at each power cycle.

Conventional application of memory BIST includes addition of a dedicated BIST controller per a group of memories. However, adoption of such an approach in complex designs (such as processor cores) which have densely packed memories operating at giga-hertz frequency range could be highly design intrusive. It could have a dire impact on functional performance and further add undue complications in design implementation (timing, routing etc.). To avoid such challenges, high performance applications which have a higher level of interface to a collection of memories with critical functional path performance typically adopt a shared bus architecture [6].

A. Shared Bus Architecture Overview

Shared bus architecture (Fig. 1) provides a shared interface which contains a common access port to a group of memories. A logical memory (LM) is an abstraction level with common address space containing one or more physical memories. A cluster is a collection of LMs which can be accessed individually through a shared interface at the cluster level. Access to memory data, control and clock ports and other controls to address specific memories inside each cluster is provided through the shared interface. Description of memory cluster module, shared bus interface ports, logical and physical memory information is provided inside library files. EDA (Electronic design automation) tools use this information to insert the BIST controller at cluster level.

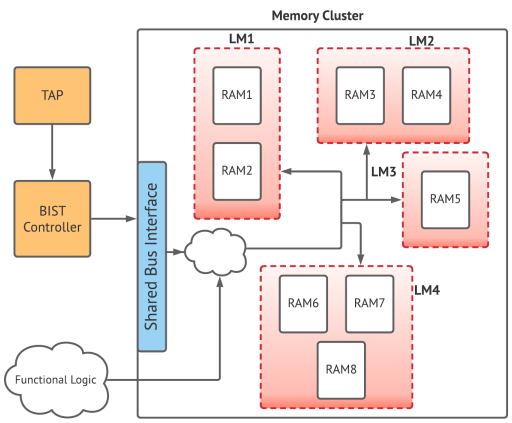


Fig. 1. Illustration of shared bus architecture

For shared bus implementation, the BIRA does repairability analysis for one LM at a time independently and the repair solution is applied to the PMs associated with the LM through BISR simultaneously.

Usually complex memory systems such as L1/L2 data and instruction cache in high performance processor cores could contain multiple instances of memories which are interlinked. General practice involves packing these together and placing them in a confined region in the chip floor plan to optimize overall space utilization. However, the downside of such an approach is that the complexity of timing closure and routing of signals to and from the individual memories significantly rises as the number of instances scales up. To overcome these challenges, designers normally tend to replace a bunch of smaller physical memory instances with a single larger composite memory instance with additional external decoding for data/address channels to enable memory access arbitration. This technique aids in significant area optimization and also eases timing closure by virtue of reduced memory access time.

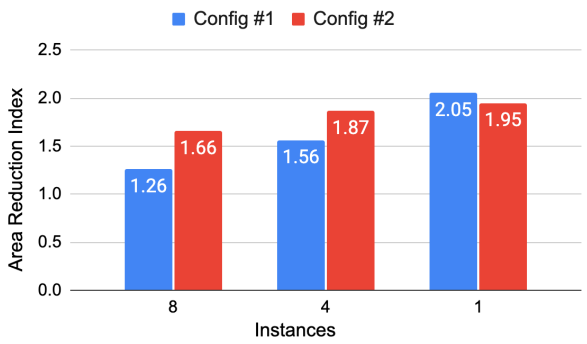
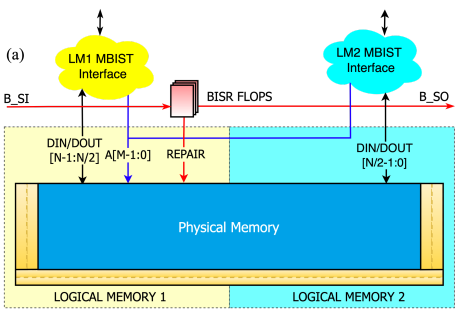


Fig. 2. Area reduction across memory splits.

For instance, consider two different configurations of a memory cluster containing 16 memory instances. Figure 3 shows the area reduction benefit that could be obtained by implementing it using fewer memory instances (such as 8, 4 and 1 instances) of larger size and similar type on a sub-10 nm technology node. As it can be seen, implementing the cluster with just one physical instance as compared to 16 independent slices of memories could help gain almost ~2X area reduction.

In the context of shared bus implementation, the sharing of physical memories can extend across two logical memories through a two way access mechanism using the left right enable system considering a center decode based memory instance. A basic scenario of a two and four way horizontal split of a single PM based on data width across two and four LMs is shown in Fig. 3(a) and 3(b) respectively.



Authorized licensed use limited to: Uppsala Universitetsbibliotek. Downloaded on April 06,2026 at 14:30:24 UTC from IEEE Xplore. Restrictions apply.

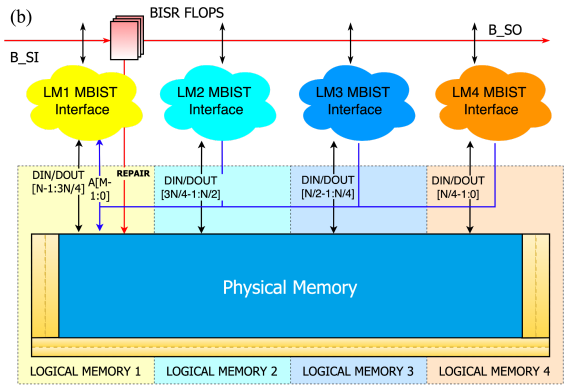


Fig. 3. Shared PM scenarios.

B. Limitations of Shared PM

While this approach works seamlessly in case of regular non-overlapped scenarios, it imposes few challenges and limitations for schemes which share a single PM across multiple LMs.

Firstly, the repair information of PM can be exported only to one LM. As a consequence, this could leave a vulnerability of defects in the section of memories covered by other LMs unrepairable, which can result in potential yield loss. For future reference, we denote the redundancy configuration of a PM as *{n}C{y}R* where n and y are the number of spare columns and rows available for repair respectively.

a) Dual redundancy configuration

Consider a *2C2R* memory configuration with two spare columns equally split between two halves (one per each bank) and two spare rows which are common to both. The limitation of repair gap on the second LM can be partially mitigated by assigning one set of spare columns per each LM mapped to each bank at the cost of reduced redundancy. However, the same cannot be done with two spare rows since there is a possibility that a common address location could be identified as the failing row address (FRA1 and FRA2) for both LMs, thereby resulting in an invalid condition and corrupting the memory access.

b) Single redundancy configuration

For scenarios such as *1C* and *1C1R*, the redundant elements of PM can be mapped to only one LM which could enable coverage for only half of the memory leaving the other half exposed. Secondly, repair sharing cannot be extended across logical memories as BISR typically handles each LM independently.

III. PROPOSED METHODS

In this section, we present two novel strategies to ensure full repair coverage in case of shared PM scenarios. The proposed solutions enable sharing (or distribution) of available redundant elements within the PM across multiple LMs using custom modifications. The methodology allows assigning independent repair configuration mapping and analysis for each LM corresponding to a PM, enabling BISR insertion per LM level separately. Custom arbitration logic is added outside the PM (within a wrapper) to map LM level repair implementation to PM level to utilize the spare elements optimally while avoiding potential conflict conditions.

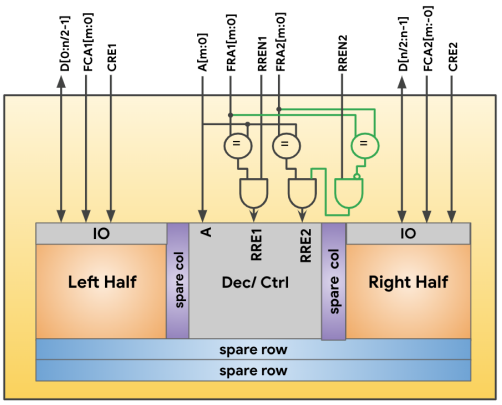


Fig. 4. Basic custom wrapper method.

A. Basic Custom Wrapper Solution

As summarized in Section II.B, a simple solution in case of two LMs implemented using a single PM with *2C2R* configuration would be to assign one spare column from each bank to each LM. However, the same cannot be done with the 2 rows. Since the repair analysis is independently done for each LM, the possibility of potential conflict scenario *FRA1=FRA2* could create an invalid condition corrupting the memory access. To mitigate this, a simple tactical solution to check for equivalence (as shown in Fig. 4) is implemented in the memory wrapper to avoid the conflict condition. This allows splitting one row per each LM. In a normal use case, any faults that need to be repaired with row elements can utilize both rows as necessary.

In the event of a conflict situation, the additional gating implemented on *RREN2* signal based on {*FRA1=FRA2*} comparator check ensures that both row repair enable controls at PM level (*RRE1* and *RRE2*) are never asserted at the same time during memory access. To illustrate further, consider the following two scenarios possible in case of a PM with 2^m rows shared by two LMs.

Scenario 1: *Single faulty row 'k' is detected in either the left half or right half.*

a. *FRA1=A[k]* or *FRA2=A[k]*
b. *RRE1*=1 and *RRE2*=0 or *RRE1*=0 and *RRE2*=1

Scenario 2: *Two faulty rows detected as row 'k' in left half and row 'l' in right half.*

a. *FRA1=A[k], FRA2=A[l]*
b. If *'k'* ≠ *'l'*, *RRE1*=1, *RRE2*=1,
c. *'k'* = *'l'*, *RRE1*=1, *RRE2*=0 → *RRE1* should cover both halves since the RR is shared.

Primary benefit of this method is the improved repair coverage (covering entire PM) resulting in improved yield and thereby aiding in better cost margins. Another added benefit is that it is completely non-intrusive to critical design paths.

A shortcoming of this method is it cannot handle a multi defect scenario. For instance, two defects within a single half which would require two spare rows could not be repaired with this method as it exceeds the number of allocated spare rows to each half.

Authorized licensed use limited to: Uppsala Universitetsbibliotek. Downloaded on April 06,2026 at 14:30:24 UTC from IEEE Xplore. Restrictions apply.

B. Advanced Custom Wrapper Solution

To address the limitations of the basic wrapper solution, an advanced custom wrapper solution is proposed which gives much more flexibility in repair sharing across LMs. Instead of splitting or distributing available redundant elements per LM, this enhancement allows each LM to leverage the entire set of common redundant elements available in the PM. To accomplish this, the repair interface of PM is virtually ported to each LM to allow independent BISR insertion and repair analysis at each LM level. Subsequently the pseudo-repair ports at LM level are merged using a decoder within the memory wrapper to drive the actual repair ports of PM. Custom decoder helps in arbitration of repair access and resolve potential conflict scenarios that could arise due to different redundant elements simultaneously enabled during a single memory access (write/read). Unlike the basic solution, this method is highly scalable and enables covering multi-defect scenarios.

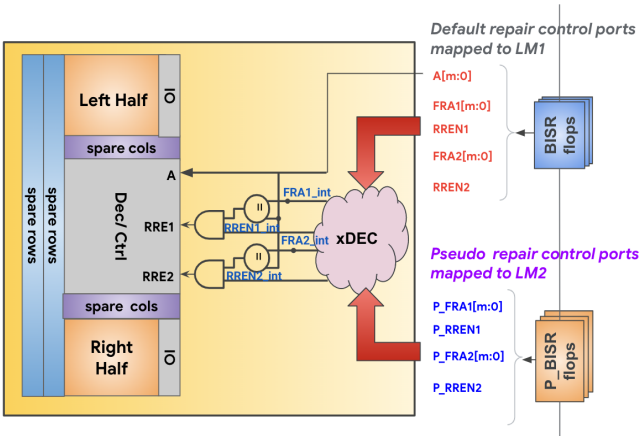


Fig. 5. Advanced custom wrapper with xDEC.

Fig. 5 shows the modification required in the memory wrapper that drives PM repair ports as a function of independently driven BISR values per LM. xDEC indicates a custom decoder that can be implemented as a soft logic to implement the conditions as listed in Table 1. The matrix captures different repair configuration scenarios and how the external BISR values are assigned to the actual repair ports of the physical memory. Basic premise here is that the row spare elements are utilized in order during repair analysis, which imply *{RREN1=0, RREN2=1}* is an invalid condition for a given LM.

Table 1. xDEC Matrix

RREN1	RREN2	P_RREN1	P_RREN2	RREN1_int	RREN2_int	FRA1_int	FRA2_int
1	1	X	X	1	1	FRA1	FRA2
0	0	1	1	1	1	P_FRA1	P_FRA2
1	0	1	X	1	'1' if (FRA1 ≠ P_FRA1) else '0'	FRA1	P_FRA1
1	0	0	0	1	0	FRA1	X
0	0	1	0	1	0	P_FRA1	X
0	0	0	0	0	0	X	X

With these conditions, following multi-defect scenarios could be effectively covered across both LMs using xDEC logic.

Scenario 1: *1 defect in each LM requiring 1 spare row for both (similar to basic solution).*

Each LM can utilize one spare row independently. In case of a conflict condition, the same spare row will be used in both LMs keeping *RREN2_int* = 0.

Scenario 2: *1 or 2 defects in LM1 requiring 1 or 2 spare rows respectively)*
PM repair ports are controlled as a function of LM1 repair ports. It's a no-conflict condition.

Scenario 3: *1 or 2 defects in LM2 requiring 1 or 2 spare rows respectively.*
PM repair ports are controlled as a function of LM2 repair ports i.e. pseudo repair ports on PM wrapper. It's a no-conflict condition.

Scenario 4: *2 defects in LM1 [row 'k', row 'l'] and 1 or 2 defects in LM2 [row 'k' and/or row 'l'] or vice-versa*
Multiple defects detected across 2 LMs actually map to 2 defects in the PM and could be repaired. This would need additional overhead in xDEC logic for comparison on *FRA** and *P_FRA**.

Advanced custom wrapper solution provides multifold benefits and flexibility with regards to enabling higher repair coverage for different flavors of memory implementation. Few differentiated advantages are listed below.

a) Improved repairability allowing the possibility of repairing multiple defects simultaneously.
b) Highly scalable. Can be implemented in any shared PM across multiple LM configurations independent of the number of redundant elements available within the PM.
c) Completely non-intrusive to design performance.
d) xDEC implementation can be simplified or extended based on the use case of single or multiple shared PMs scenarios.
e) Very minimal area overhead as compared to the option of splitting the PM (<0.01% for 2 clusters with 9-10% shared PMs).

Besides the benefits, there are few caveats that one needs to consider with regard to repair sharing in general in the context of shared PM use cases.
a) BIRA typically declares a memory non-repairable in the event of defects more than spare elements. Since BIRA runs independently on both LMs, assuming existence of 2 spare elements, the prior utilization of spare elements may remain unknown. This can cause incorrect non-repairability analysis.
b) While the proposed solution works for parallel BISR, it may not readily work for the serial (or internal) repair where register controlling repair signatures reside within the PM.

IV. FAULT INJECTION AND REPAIR VERIFICATION

To ensure full repair coverage using proposed methods, an exhaustive fault injection and repair verification methodology is proposed and implemented. It helps validate the detectability and repairability using BIRA with specific fault injection scenarios. The methodology is quite generic and can be adopted for any memory implementation type (both shared & non-shared bus scenarios).

Authorized licensed use limited to: Uppsala Universitetsbibliotek. Downloaded on April 06,2026 at 14:30:24 UTC from IEEE Xplore. Restrictions apply.

A. Verification Methodology

Basic requisite for repair verification flow is a standard MBIST test that can write and read 0s and 1s in all locations of the repairable memories. This is to ensure that any injected static fault (stuck-at-0 or stuck-at-1) can cause an expected fail condition. It is important to note that the fault injection is always performed at PM level whereas the MBIST test is applied at LM for shared bus architecture.

MBIST test with fault injection is expected to fail with results showing failure on fault injected location. Should the test pass, it indicates that either the test did not cover fault injected memory location (for example, a test running on limited addresses) or the injected fault doesn't conflict with the value read. The latter can be due to data inversion on the memory write or read path, which needs to be analyzed and accounted for a priori.

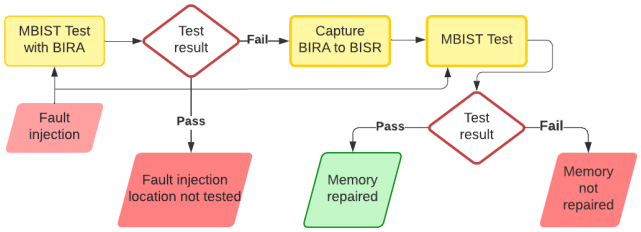


Fig. 6. Repair verification testbench

B. Fault Injection Strategy and Target Scenarios

Most memory compilers today support fault modeling which can be leveraged at the testbench level for fault injection either using parameter definitions or direct forces. Fault injection strategy is planned based on memory array dimensions (address and data widths) and availability of the spare elements.

- **Repairable fault injection:**

No. of faults injected <= No. of spare elements

The fault locations are chosen to cover the entire bit space. The expectation here is that all fault scenarios (single (Fig. 7(a)) or multiple (Fig. 7(b,c)) are repairable. Multiple fault injection enforces that all spare elements are utilized.

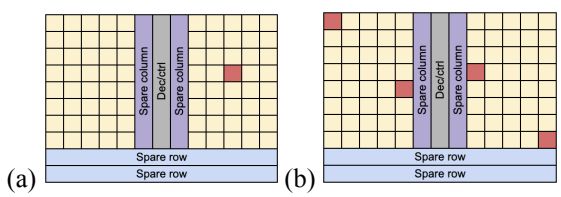
- **Non-repairable fault injection:**

No. of faults injected > No. of spare elements

This scenario ensures that repair infrastructure doesn't give any false pass results (Fig. 7(d)).

- **Special case of Shared PM:**

The reassigned distribution of spare elements as per the solutions shared in Sections III.A and III.B are considered for verification (Fig. 7(e)). Furthermore, to sensitize potential conflict conditions with the proposed solutions, addition verification is performed by injecting multiple faults across halves on the same address.



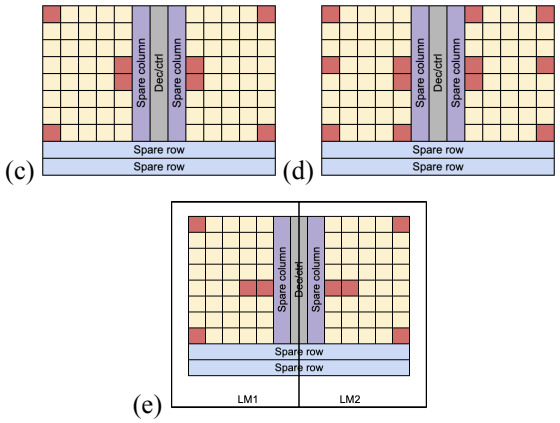


Fig. 7. Fault injection scenarios (a) single fault (b) multiple random fault (c) multiple random faults targeting row repair (d) non-repairable (e) shared PM scenario

V. RESULTS

In this section, we share results from applying the proposed method of redundancy sharing schemes on two sample memory clusters implemented using the shared bus interface and have shared PM across multiple LM use cases in a complex (sub <10nm) SoC.

Firstly, to illustrate the probability of defect occurrence in the chip, we use the equation from [12].

$$P(x) = \frac{\Gamma(\alpha+x)}{x\Gamma(x)} \cdot \frac{(Ad/\alpha)^x}{(1+Ad/\alpha)^{\alpha+x}} \qquad (1)$$

where
x = number of defects, α = clustering parameter,
Γ = gamma function, A = chip area, d = defect density.

If α=0, $P(x)$ is a delta function (maximum clustering).
α=∞, $P(x)$ is Poisson distribution (no clustering).

For a hypothetical design use case scenario of A = 200mm^2 and A = 100 mm^2, d = 0.1cm^2 and d = 1 cm^2 respectively and α = 0.5, we compute and analyze the defect probability across different defect scenarios (ranging from 0 to 10).

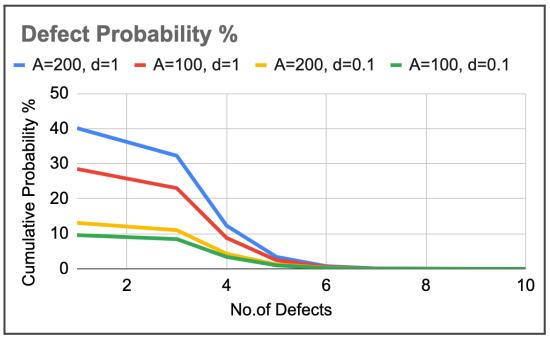


Fig. 8. Defect probability distribution

The plot (Fig. 8) based on eq(1) shows that the probability of multiple defect occurrences dip sharply with an increase in the number of defects considered. For larger designs the margin is slightly higher considering the larger critical area. Given the generalization, for practical purposes we could consider a memory totally repairable considering *2C2R* configuration for a number of multi-defects less than 4-5 [13]. Theoretically with an optimistic assumption (for the purpose of relative comparison) that all defects are in

Authorized licensed use limited to: Uppsala Universitetsbibliotek. Downloaded on April 06,2026 at 14:30:24 UTC from IEEE Xplore. Restrictions apply.

memories and all memories are repairable would imply 100% yield.

Therefore, to evaluate and assess the impact of the proposed methods we use the repairability metric of the memories to compare the relative gain pre and post the redundancy sharing implementation for shared PM scenarios.

Table 2. Repairability gap with shared PM

C#	*#LM*	*#PM*	*#SPM*	*#SPM %*	*RDN*
1	70	231	38	16.45	Col
2	34	304	96	31.58	Row+Col

C#	*#total_bc*	*#shared_pm_bc*	*#unrep_bc*	*#rep_gap %*
1	5318656	686080	343040	6.45
2	84312064	8388608	4194304	4.97

Table 2 summarizes the statistics of total number of logical memories (*#LM*) and physical memories (*#PM*), instances of shared physical memory (*#SPM*) that are shared across multiple logical memories for 2 sample memory clusters (*CSR*) along with redundancy (*RDN*) scheme used. The *#total_bc, #shared_pm_bc and #unrep_bc* statistics correspond to the total bit count, bit count contributed by shared PM and the bit count unrepairable due to shared PM respectively for each cluster. The unrepairable bit count is basically equal to the half of the PM bit count which would remain unrepairable without proposed solutions and hence the total *#unrep_bc* is half of the total shared PM bit count. It may be noted that ~5 to 6.5% (*#rep_gap*) of total bits (#total_bc) of these two clusters are unrepairable owing to shared PM implementation, which can be directly linked to the potential yield impact (under the previous assumption).

Additionally, the solution proposed have a very minimal area overhead as compared to the option of splitting the PM (<0.01% for 2 clusters with 9-10% shared PMs).

The implementation of proposed solutions (basic wrapper scheme for cluster 1 and advanced wrapper scheme for cluster 2) helped bridge the repairability gap and ensured 100% repairability of the entire memory space.

VI. CONCLUSION AND FUTURE WORK

In this paper, we focused on methods to address the repairability gaps seen in shared bus architecture with shared PM implementation by using simple custom redundancy sharing schemes. General redundancy schemes and different flavors of shared PM implementations were discussed with examples.

We proposed two novel strategic solutions to ensure full repair coverage in case of shared PM across multiple LMs scenarios by enabling sharing of available redundant elements within the PM across multiple LMs. To support multi-defect scenarios and avoid potential conflict due to repair sharing, an advanced solution with a custom voting based decoded logic was proposed. Furthermore, a comprehensive fault injection and repair verification methodology was shared which helps in testing the efficacy of proposed methods and validate on-chip redundancy analysis hardware. Results from two sample memory clusters with shared PM implementation on a large design indicate a repairability gap of ~5-6.5% being bridged by virtue of applying the proposed methods.

REFERENCES

[1] "International technology roadmap for semiconductors (ITRS) 2015 edition", Semiconductor Industry Association, June, 2015.

[2] Y. Zorian and S. Shoukourian, "Embedded-memory test and repair: Infrastructure IP for SoC yield", IEEE Design & Test of Computers, vol. 20, pp. 58-66, May-June 2003.

[3] I. Kim, Y. Zorian, G. Komoriya, H. Pham, F. P. Higgins and J. L. Lweandowski, "Built in self repair for embedded high density SRAM", Proc. Int. Test Conf. (ITC), pp. 1112-1119, Oct. 1998.

[4] M. A. Masashi Horiguchi, Jun Etoh and K. Itoh, "Improving Memory Repair by Selective Row Partitioning", Proceedings. 24th IEEE International Syposium on Defect and Fault Tolerance in VLSI Systems, pp. 211-219, 2009.

[5] J.-F. Li, J.-C. Yeh, R.-F. Huang, C.-W. Wu, P.-Y. Tsai, A. Hsu, et al., "A built-in self-repair scheme for semiconductor memories with 2-D redundancy", Proc. Int. Test Conf. (ITC), Sept. 2003.

[6] T. McLaurin, F. Frederick and R. Slobodnik, "The DFT challenges and solutions for the ARM® Cortex™-A15 Microprocessor," IEEE International Test Conference, 2012.

[7] C.-T. Huang, C.-F. Wu, Li and C.-W. Wu, "Built-in redundancy analysis for memory yield improvement", IEEE Trans. Rel., vol. 52, no. 4, pp. 386-399, Dec. 2003

[8] P. Öhler, S. Hellebrand and H.-J. Wunderlich, "An integrated built-in test and repair approach for memories with 2D redundancy", Proc. 12th IEEE Eur. Test Symp., pp. 91-96, May 2007.

[9] W. Jeong, J. Lee, T. Han, K. Lee and S. Kang, "An advanced BIRA for memories with an optimal repair rate and fast analysis speed by using a branch analyzer", IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 29, no. 12, pp. 2014-2026, Dec. 2010.

[10] K. Amirkhanyan, S. Shoukourian and V. Vardanian, "A mechanism of row redundancy sharing for reparable memory systems," 2017 Computer Science and Information Technologies (CSIT), 2017.

[11] T. Kawagoe, J. Ohtani, M. Niiro, T. Ooishi, M. Hamada and H. Hidaka, "A built-in self-repair analyzer (CRESTA) for embedded DRAMs", Proc. Int. Test Conf., pp. 567-574, Oct. 2000.

[12] Akhil Garg and Prashant Dubey, "Fuse Area Reduction based on Quantitative Yield Analysis and Effective Chip Cost", Proceedings of the 21st IEEE International Symposium on Defect and Fault-Tolerance in VLSI Systems (DFT '06).

[13] G. Wang and C. Chang, "Design and implementation of shared BISR for RAMs: A case study", IEEE Autotestcon Anaheim CA, pp. 1-7, 2016.

Authorized licensed use limited to: Uppsala Universitetsbibliotek. Downloaded on April 06,2026 at 14:30:24 UTC from IEEE Xplore. Restrictions apply.